

— TEST-TIME COMPUTE · 检索 · 冻结模型

×

搜索，是一种 test-time compute

与其把模型换大，不如在推理这一刻
多算几遍

肖涵

Elastic 副总裁



微信



模型这边

榜单上挤满了人，第一名和第十名差几个千分点。

换谁家的 embedding，效果都差不多。

链路这边

召回、精排、混合检索、查询改写——该有的组件，十年前就都有了。

于是大家的结论

搜索是个成熟行业，剩下的都是工程活，值得投入的地方在别的方向上。

这几个月我一直在想这件事。
后来发现，我们可能一直站在一个不太对的角度看它。



搜索，本来就是一种 test-time compute。

看清这一点，很多原本觉得做到头了的地方，会重新变得有事可做。

一句话：不去训一个更大的模型，而是让手上这个模型，在回答每个问题时多干一点活。训练阶段的钱已经花完了，这笔新的开销花在推理这一刻。

Best-of-N

采样多个候选答案，选出其中最好的一个。

Self-consistency

让模型把推理过程走多遍，取出现次数最多的答案。

Verifier search

用一个判分器在候选里搜索，投入的算力越多，结果越好。

模型没变，权重一个没动。变的只是你愿意为这一次回答，多花多少算力。

让机器人在一手扑克上多想 **20 秒**，effect 相当于把模型放大 **10 万倍**、再多训 10 万倍的时间。

Noam Brown, OpenAI, 2024

20 秒，换 10 万倍。

这个兑换比例一旦成立，问题就从「模型够不够大」，变成了「你舍不舍得在推理时多花这**一点**」。



01 | 为什么说搜索就是它

多跑几遍，跑得更深一点，然后看看能换回来什么

向量召回一遍，精排再来一遍，查询改写之后又是一遍，多路结果还要融合。这些环节没有一个是
在训练阶段完成的，**全都发生在用户按下回车之后**。这就是 test-time compute，只是我们一直管它
叫「搭链路」。

A

Multi-pass: 多跑几遍

同一个模型，换个角度再编码一次。把文档切成句子分别算，把图片切成局部分别看。**每一遍都会带回上一遍漏掉的东西。**

B

Deeper pass: 跑得更深

同一次前向，不只取最后那个池化向量，把中间的 patch、token 全都读出来用。**算力几乎没多花，信息多拿了一大截。**

C

Compose: 串起来

让 agent 自己决定先 grep 还是先 embed，要不要重排，要不要再搜一轮。**链路本身成了推理期的产物。**

单向量 COSINE



$$\cos(q, d)$$

一篇文档压成一个向量

起点

句子级 MAXSIM

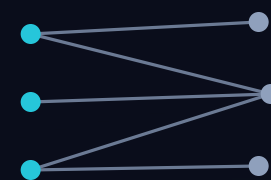


$$\max_s \cos(q, s)$$

还是那个模型，把文档切成句子各算一遍

这一步就是 TEST-TIME COMPUTE

COLBERT 多向量



$$\sum_i \max_j \cos(q_i, d_j)$$

每个 query token 对全部 doc token 取 max

得换一个多向量模型

中间这一列最有意思：模型一个字节都没改，只是在推理时多算了几遍，就走到了原本得换模型才够得着的位置。



你花出去的
推理时间

multi-pass · deeper pass · 更长的 agent 循环

买

01 相关性

同一个任务，做得更准

02 新能力

模型压根没被训练过的事，它也能干

搜索的上限不取决于模型有多大，
而取决于你愿意为一次查询付出多少推理算力。

01

重排与页内检索

召回之后再上一个模型，把候选放在一起互相比。极端情况下，索引干脆可以不要。

买到的是 · 精度

02

图像标注

一个只会做检索的模型，被推理期的算力改造成了开放词表的标注器。

买到的是 · 新能力

03

Dataroom

用便宜的本地小模型把开放网络啃成一个带引用的 zip，留着贵的 token 干正事。

买到的是 · 召回

04

Searchbox + 知识图谱

把 agent 关进没有网的盒子里，只能自己现搭链路。再用知识图谱给它出多跳难题。

买到的是 · 精度

从模型内部一直做到 agent 链路，代码全部开源。



02 | 最熟的那笔算力：重排

召回给你一堆候选，重排再花一次钱，把它们放在一起比一遍

向量检索里，每篇文档的向量是**在别的文档还不存在的时候**就算好的。可文档相关不相关，本来就是比出来的：三段都提到延迟，哪一段真的回答了问题？重排就是为这件事额外花的一次推理。

召回阶段

每篇文档一个向量，离线算好。**便宜，能扫全库，但每篇都是盲评。**

重排阶段

query 和最多 64 篇候选塞进同一个 131K 上下文，跨文档做自注意力，在每篇的最后一个 token 上读出分数。**每篇都是在跟对手比。**

jina-reranker-v3, 0.6B

61.94

BEIR nDCG@10。同样 0.6B 的 bge-reranker-v2-m3 是 56.51，Qwen3-Reranker-0.6B 是 56.28

0.6B

比生成式的 listwise 重排小一个数量级，却拿到更好的 BEIR

64 篇

一次调用里同时参与比较的候选数量

只有一页文档时，把索引删掉



GITHUB



常规做法是把每个 chunk 都 embed、存向量、再用 ANN 查回来。但索引是靠很多次查询摊薄成本的，而一页文档只活在这一次浏览里。为它建索引，比直接跑一遍前向还贵。

所以这里反过来：**把整页塞进一次 listwise 前向**，答案直接以 <mark> 标回原文，页面自己的 CSS、表格、图都不动。

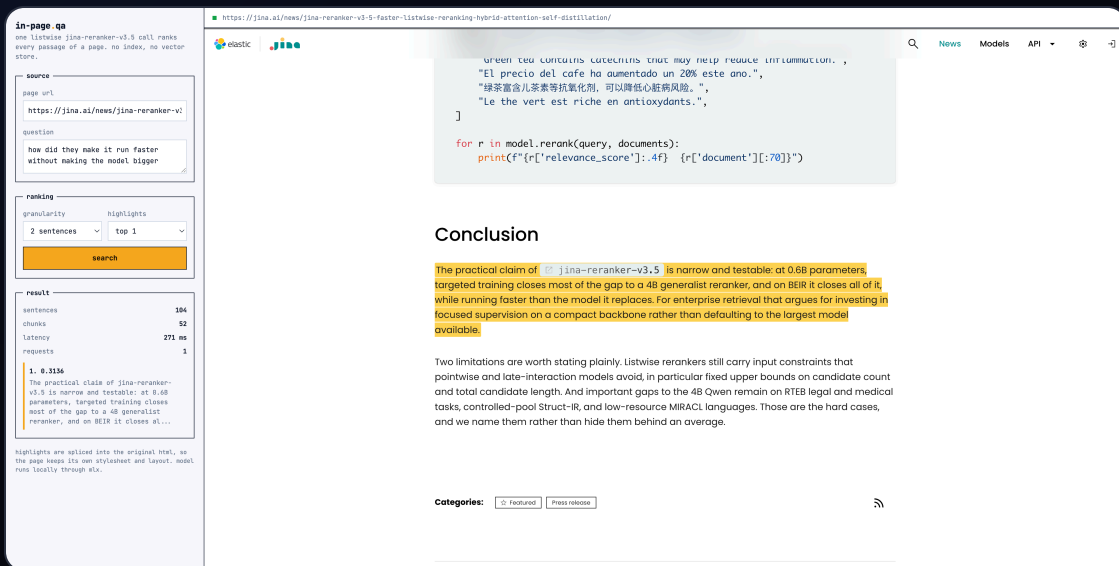
271 ms

104 句 / 52 chunk / 约 3.5K token，一次请求，M3 Ultra

0

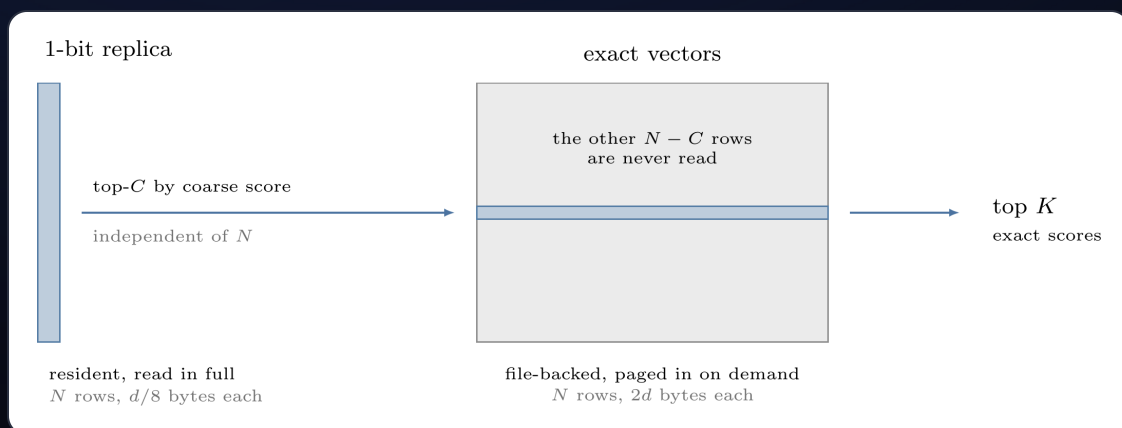
索引、向量库、chunk 数据库，一个都不需要

一百万篇的时候你当然还要第一段召回。而只有一页的时候，该删掉的恰恰是召回。



提问，答案在原页面里点亮。问题里没有一个实词出现在它找到的那句话里。

1 bit 扫全库，精确向量只读短名单



条宽是每行的字节数，条高是行数：副本只有精确向量的十六分之一宽。青色标出一次查询真正读了什么——副本全读，精确向量只读短名单那一段。

Omni 里没有 ANN 索引，一次查询要扫全库。所以同一个空间存两份：一份只保留每个坐标符号的 1 bit 副本，扫描读它；精确向量只有短名单才碰。一行 $d/8$ 字节对 bf16 的 $2d$ ，正好十六分之一。

取符号之前先做一次随机 Hadamard 旋转。旋转是正交的，**不改变任何内积，也就不改变排序**，只是把每一维携带的信息摊平到所有维度上，1 bit 才够用。查询侧不压到 1 bit：旋转后拆成**四个位平面**，Metal kernel 用几次 popcount 给一行打分。

粗排分数只决定谁进短名单，所以它不必准，只要短名单够宽。**最终排序由精确重算说了算，成本只跟短名单长度成正比**。候选集是 $\min(4096, \max(1024, \text{topK} \times 32))$ 。

1.76x

50 万行时相对全量精扫的加速，M3 Pro 14 核

9.7 ms

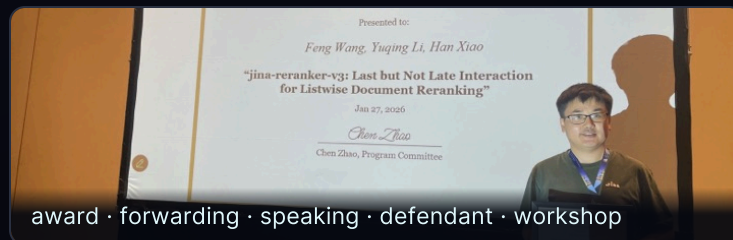
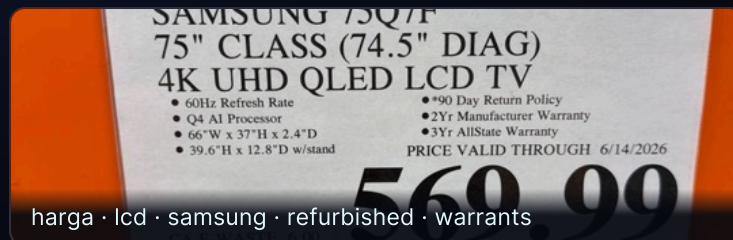
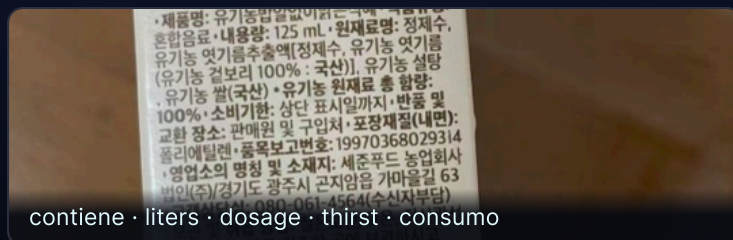
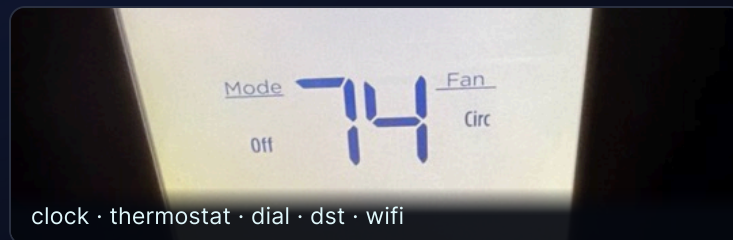
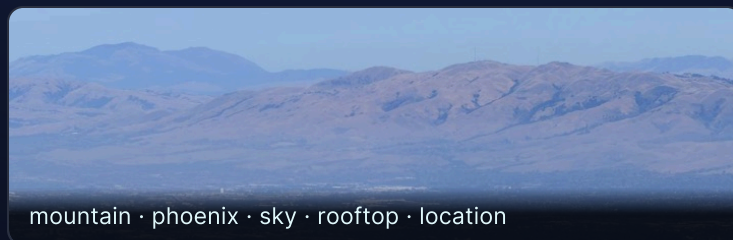
文本查询 p50，M3 Ultra，真实个人文件库



03 | Image tagging: 买一个新能力

一个只会做检索的模型，从来没学过打标签。这次不求它更准，求它会一件新事

这些标签出自一个检索模型



没训练、没请第二个模型、没查词典，全部由推理期的计算得到，而且天然是多语言的。

- | 手上有 一个 **jina-embeddings-v5-omni-nano**，大约 1B，图和文共用一个 768 维空间。
- | 要它干 把图里**每一个**东西都说出来：多标签，而且词表是开放的。
- | 前提是 权重全冻结。它是个检索编码器，**没有标注头，没有分类器，训练时压根没见过这个任务。**
- | 只准用 它自己吐出来的那些数，在上面做推理期计算。别的一概不行。

不训练

不请第二个模型

不用 WordNet 和词典

不用正则和词性标注

前面几个项目都是把它会的事做得更好，这一次是让它做一件根本不会的事。

A**同一次前向，读得更深**

别只拿那个池化完的向量，把 **patch 级** 的输出全读出来，挨个跟 12.8 万个词打分。
算力几乎没多花。

几乎不额外花钱

B**换个视角，再跑一遍**

把图切成若干 crop **重新编码**。小物体在自己那张 crop 里占满画面，不再被整图的池化冲淡。

算力随视角数线性增长

C**在已有结果上再做文章**

白化、最优传输、图传播、EM——在**同一批像素**上做更花哨的数学。

算力全花在数学上

下面几页逐个看这三种花法各自换回了什么。

词表当标签空间



每个 patch 打分



减掉背景先验



去重取头部

01 tokenizer 的词表，直接拿来当标签空间

12.8 万个 token 全部编码成文本向量。不用人工整理标签集——模型认识多少词，就有多少个候选标签，开放词表不需要额外代价。

02 每个 patch 单独打分，再跟全局向量融合

$0.7 \times \text{patch-max} + 0.3 \times \text{global}$ 。小物体不再被整张图的池化冲淡，mAP 从 0.264 直接到 0.635。

03 每个词减掉它自己的先验

高频词与任何一张图的相似度都偏高。减掉 μ_l 之后，剩下的右尾才是真正的证据。

04 词首过滤，再在向量空间里做 NMS

12.8 万压到 2.5 万候选；然后把 kitty / cat / kitten / chatte 这种同义词簇合并成一个概念。

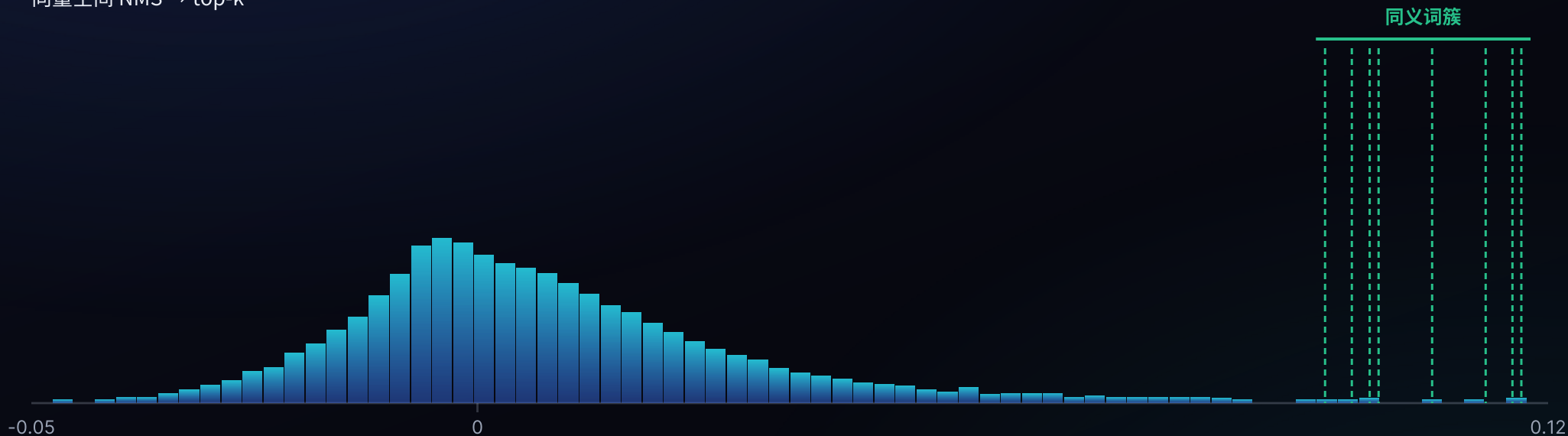


12.8 万个词的得分如何被整理出来

4 · 右尾去重

向量空间 NMS → top-k

token 数 (√ 尺度)



NMS 之前的 top-8: kitty · cat · kitten · chatte · kitt · cats · kittens · cosy, 其实是一个概念

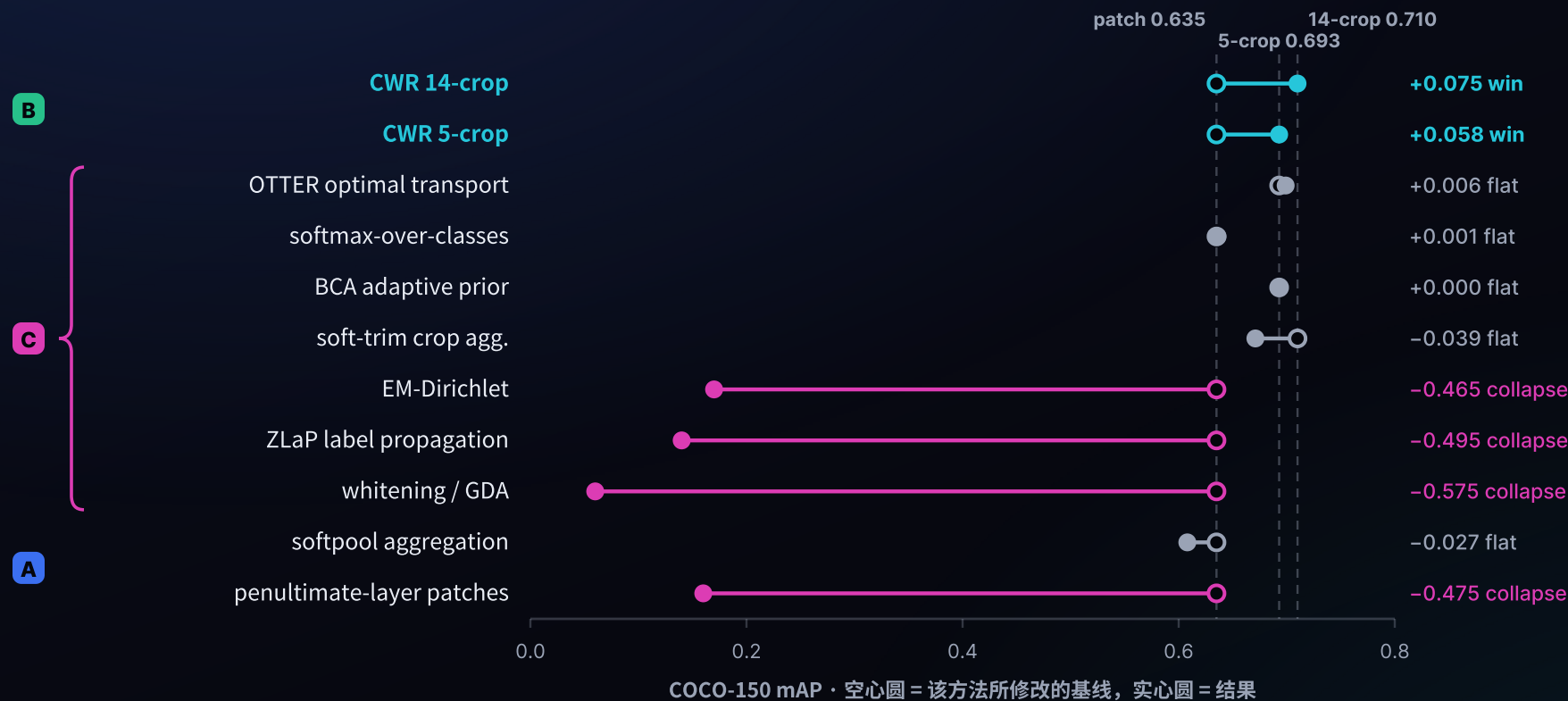
NMS 之后 → kitty · cosy · plush · crib · paw · sleeps



做法	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

只用池化向量是 0.264；改读 patch 之后 0.635；再多切几个 crop 重新编码，0.710。

A 类一步就 +0.371，B 类再补 +0.075。这中间模型一个权重都没动过。



能站住的是 B 类：真的去看新的像素。A 类把这次前向读透。

至于 C 类那些更花哨的数学——它们的前提是图和文在两个空间里，可这儿本来就是一个空间。



能带回新信息的算力才换得到精度



75 毫秒读透一次前向，1 秒钟看十五个视角。这条前沿线上的每一步，都是往系统里灌进了新东西。

+1.3 ms

每张图打标签的额外耗时，约为编码成本的 3%，搭在原本就要跑的前向上顺手完成

0.847

精细模式（5-crop）P@1，Swift/bf16 端口，基线 0.773

32 帧

视频每 240 秒采样一次，跨空间和时间一起取 max

图片

建索引的时候顺手把标签也产出来，不用第二次前向。

视频

抽帧之后共用同一套 patch 打分逻辑，时间维直接并进 max。

扫描版 PDF

当图片处理。那些 OCR 认不出来的老档案，现在也能用词搜到了。

编码器、索引、存储全在那台存着文件的 Mac 上。从一篇实验记录到一个能用的功能，中间只隔着这 3% 的算力。



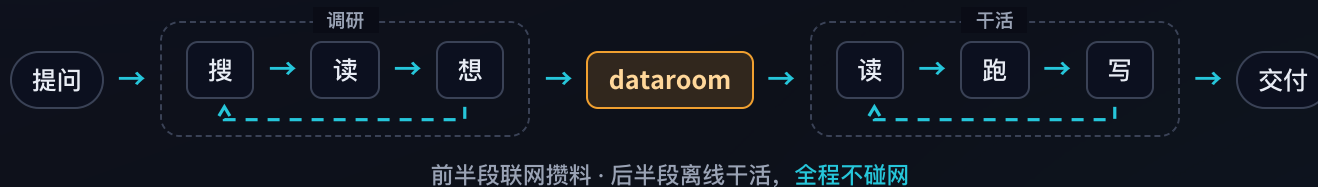
04 | 往上一层：让 agent 自己搭链路

前面两个项目都在模型内部。这一层，算力花在了 agent 的循环里

2025 · Deep research



2026 · 长程任务



调研和干活，该用的 token 不是一种。攒料用便宜的本地小模型，贵的额度留给真正需要它的那一段。

dataroom: 把开放网络啃成一个 zip



GITHUB



给它一个 token 预算，它就开始**搜、读、写**，一轮一轮来，直到把这个题目下能找到的东西全部打包成一个**带引用的 zip**。

这个名字是我创业时留下的习惯——当年给投资人准备 data room，把该看的材料一次性理清楚放进去。现在换成给 agent 准备。

关键是 token 的账：探索阶段用自己部署的 Qwen3.6-35B-A3B、Qwen3.8-27B 跑，把昂贵的前沿额度省下来，留给后面真正需要判断力的那一步。

dataroom

攒出语料 (.zip)



searchbox

只从这个 zip 里找答案



任务实时页面：覆盖度进度、token 消耗、工具调用分布。停止条件看的是覆盖够不够，不是钱花完没有。

searchbox: 把 agent 关进没有网的盒子

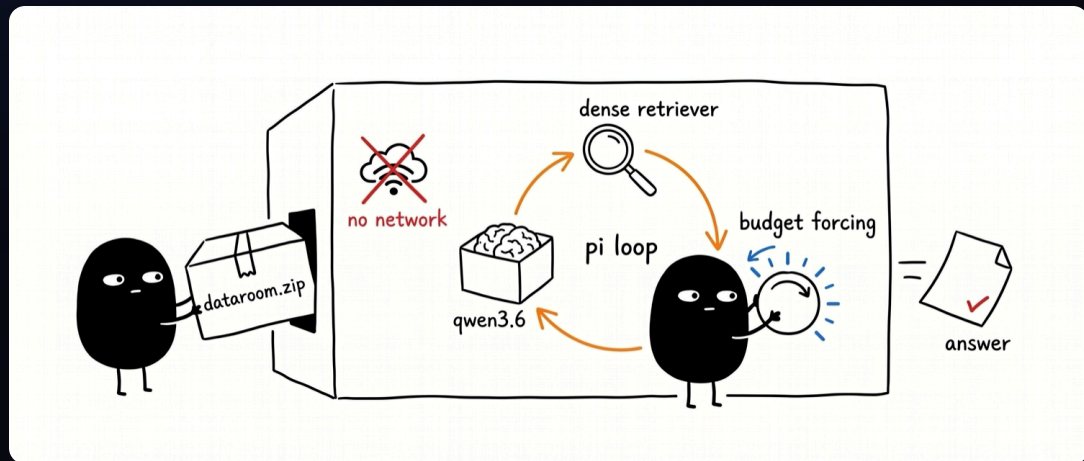


丢给它一个 dataroom 的 zip，然后**切断网络**，再问一个问题。自己部署的 Qwen3.6-35B-A3B 在里面跑。它想答上来，只能**用手边的本地工具现搭一条链路**——grep、embed、rerank、相似度、聚类、选多样性。

断网这件事是故意的。**不断网，就分不清它是真的在检索，还是网上碰巧有现成答案。**

正在研究的几个问题

- 它第一个伸手去拿的是哪个工具？
- 是不是 grep 就够了，稠密检索到底在哪些地方才真有用？
- 逼它多花算力，难题上会不会有起色？
- 它搭出来的链路，下次还能不能复用？



断网循环，画成漫画就是这样

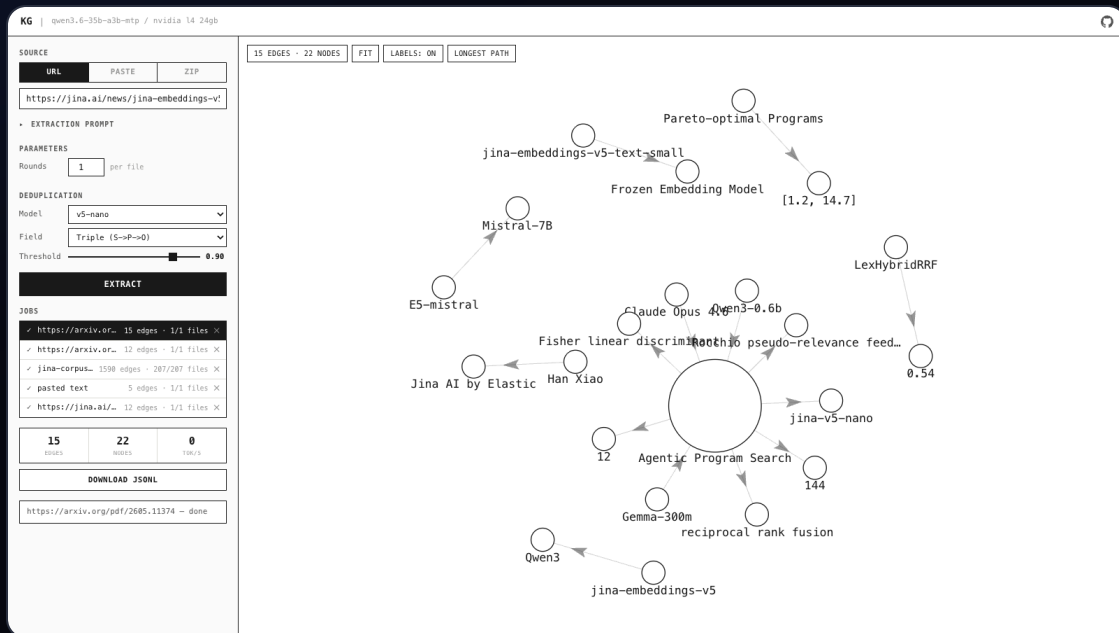


简单题对研究 agent 搜索毫无用处：**一次 grep 就能命中的问题，什么方法做出来都一样分。**要评测 searchbox，得先有真正需要搜索才能答的题。

怎么造

把语料先抽成知识图谱，每条事实是一条 (主语)-[谓语]->(宾语) 的边，然后去**走图里最长的那些路径**。这些长链条自然就成了**多跳问题**——没有任何单独一段文字能直接回答。

题目从同一份语料里长出来，所以是**私有的、不可能被背过的评测集**。agent 必须真的把事实一环一环连起来，也就必须真的去花那份推理算力。



实时图谱：每条事实一条边，最长路径视图会把多跳问题直接挑出来。

前面三个是完整的链路。想知道**检索这一环到底值多少**，得把它单独拎出来：固定一份 Jina 文档语料（210 篇 md 加结构化 json，切成 7777 个 chunk），外面套一个 agent，然后**只给它一个外部工具**。

其余全是 Pi 自带的 read、grep、bash。这样被测的就只有 search_corpus 这一件事，**好和坏都只能归给它**。

命中不是答案

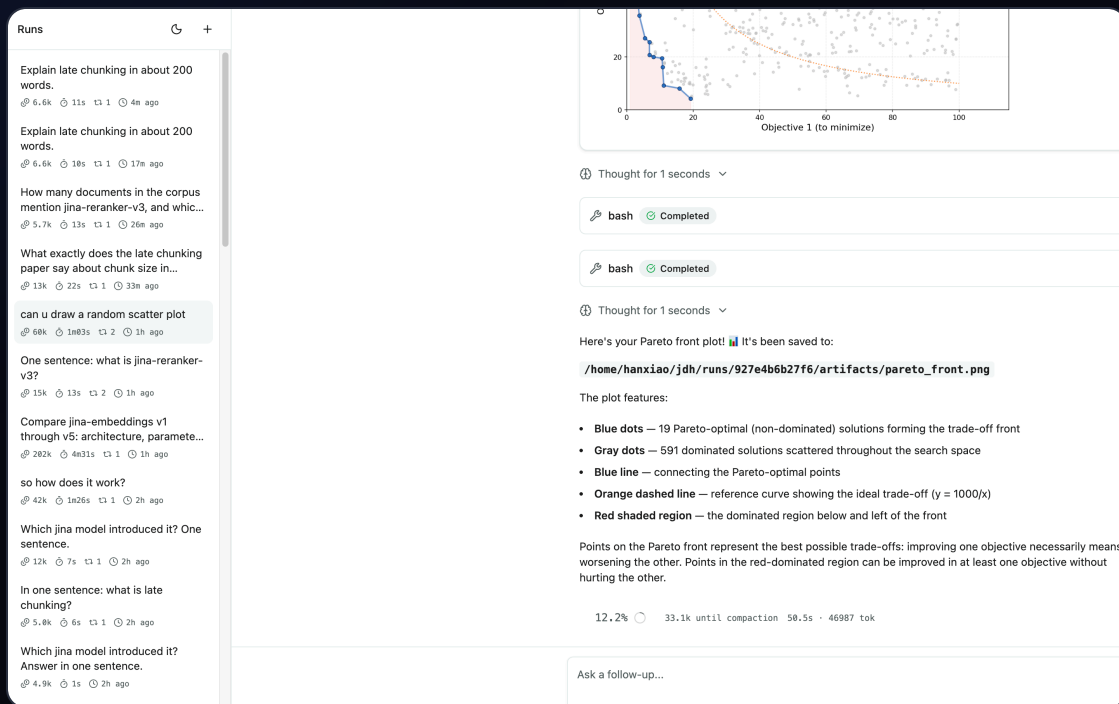
返回 {score, path, lines, text}，agent 得自己回原文把段落读完、再 grep 一遍看还有哪儿提到。

没有 SIDECAR

7777×768 的矩阵直接在扩展里算：加载 117ms，单次检索 4.3ms。少一个要启动、要占端口、会挂掉的服务。

联网是开关

默认关。开了才挂上 search_web 和 read_url，于是「翻书」和「上网」可以分开计分。



跑完一轮的样子：思考流、每一次工具调用及其参数、上下文占用、自动压缩、每段花了多少秒，全部流式开着。

01

dataroom · 拉满召回

便宜的本地 token 把网络啃成一个带引用的 zip。这一步决定了天花板——材料里没有的东西，后面谁也变不出来。

02

searchbox · 拉满精度

断网，只给本地工具，逼 agent 在推理期自己组装链路。这一步决定了你能从材料里捞出多少。

03

知识图谱 · 说了算的裁判

从同一份语料长出多跳难题。有了它，前面两步的每一次改动才有得比。

04

harness · 单独衡量检索

固定语料，只留一个检索工具。把链路里那一环的贡献单拎出来看。

跟前面几个项目是同一件事，只是换了一层：那边组装的是模型阶段，这边组装的是工具链。都没有把模型换大。



05 | 回到那句话

这几个项目跑完，我更相信这个判断了



一层在模型里面，一层在模型外面

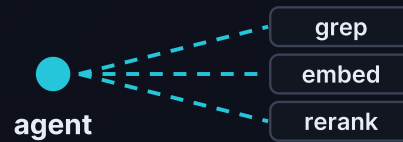


A 模型内：多算几遍

推理时干的 加一段模型，或把这次前向读透

项目 重排 · 页内检索 · 两段漏斗 · 图像标注

买到的 精度 + 一个全新的能力



B 模型外：工具链

推理时干的 自己决定先搜什么、再用什么

项目 dataroom · searchbox · 知识图谱 · harness

买到的 召回 + 精度

两层，一个赌注：多花推理算力，而不是换个更大的模型。

01 先问：这次多花的算力，带回了什么新东西？

被池化丢掉的 patch，被缩放丢掉的像素，被单向量抹平的句子结构，被一次 grep 漏掉的那几跳。能说出带回了什么，这笔钱就花得值。

02 再问：最便宜的那一档，是不是已经够了？

图像标注里，光是把这次前向读透就吃掉了 +0.371 mAP，之后十五倍的算力只再添 +0.075。先把手上这次算完的东西读干净，往往比再跑十遍划算。

03 最后问：这笔算力，该花在哪一段？

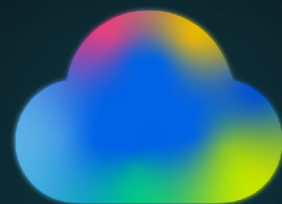
一页文档不必建索引，一百万篇仍然需要第一段召回；攒料用便宜的本地 token，判断留给昂贵的模型。同样一笔钱，花错地方就是白花。

按这三条走，2026 年的搜索远远没到没事可做的地步。



搜索是一种 **test-time compute**。
别急着换更大的模型，
先在推理时**多搜一点**。

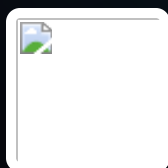
这几个月我一直在这条路上，
它比我想的要宽。



THANKS

肖涵 · Elastic 副总裁

微信



IN-PAGE SEARCH

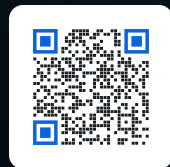
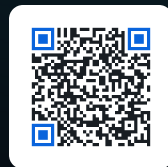
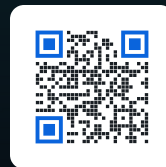


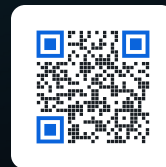
IMAGE TAGGING



DATAROOM



SEARCHBOX



KNOWLEDGE-GRAPH

